COMSATS UNIVERSITY ISLAMABAD , ATTOCK CAMPUS

Information Security

Name:

M. Haider Iqbal

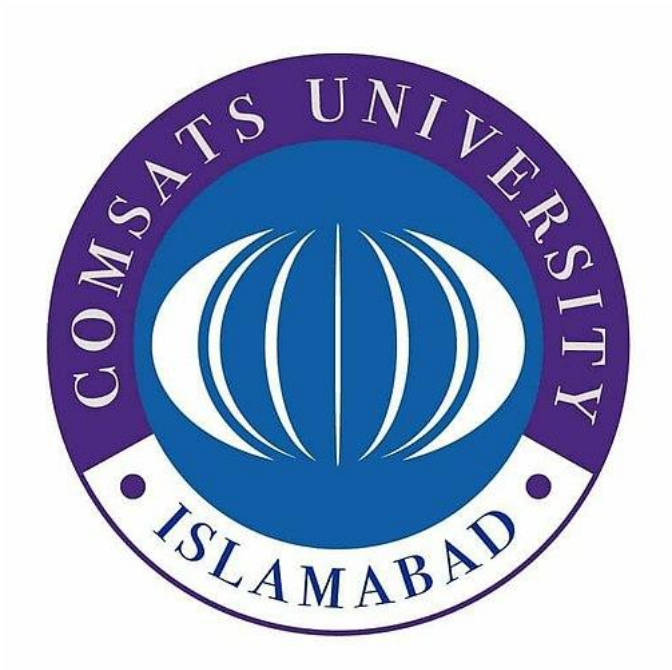
Reg No:

SP24-BSE-039

Submitted to:

Ms. Ambreen Gul

Lab Terminal:



QUESTION NO 1:

CODE :

```
# ============================================================
# Secure Email Communication System
# RSA Encryption + ElGamal Digital Signature
# ============================================================

import random
import math

# ------------------------------------------------------------
# Utility Functions
# ------------------------------------------------------------

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def mod_inverse(a, m):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

# ------------------------------------------------------------
# RSA SECTION (CONFIDENTIALITY)
# ------------------------------------------------------------

def rsa_key_generation():
    # Small primes for demonstration only
    p = 61
    q = 53

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 17
    d = mod_inverse(e, phi)
```

```
    return (n, e), (n, d)

def rsa_encrypt(message, public_key):
    n, e = public_key
    ciphertext = []

    for char in message:
        ciphertext.append(pow(ord(char), e, n))

    return ciphertext

def rsa_decrypt(ciphertext, private_key):
    n, d = private_key
    plaintext = ""

    for value in ciphertext:
        plaintext += chr(pow(value, d, n))

    return plaintext

# ------------------------------------------------------
# ELGAMAL SECTION (INTEGRITY & AUTHENTICITY)
# ------------------------------------------------------

def elgamal_key_generation():
    p = 467         # Large prime (small here for demo)
    g = 2           # Generator

    x = random.randint(1, p - 2)  # Private key
    y = pow(g, x, p)              # Public key

    return (p, g, y), x

def elgamal_sign(message, private_key, public_params):
    p, g, _ = public_params
    x = private_key

    # Simple message hash
    m = sum(ord(c) for c in message) % p
```

```
    while True:
        k = random.randint(1, p - 2)
        if gcd(k, p - 1) == 1:
            break

    r = pow(g, k, p)
    k_inv = mod_inverse(k, p - 1)
    s = (k_inv * (m - x * r)) % (p - 1)

    return (r, s)

def elgamal_verify(message, signature, public_key):
    p, g, y = public_key
    r, s = signature

    if r <= 0 or r >= p:
        return False

    m = sum(ord(c) for c in message) % p

    left = (pow(y, r, p) * pow(r, s, p)) % p
    right = pow(g, m, p)

    return left == right

# ------------------------------------------------------
# MAIN PROGRAM (SECURE EMAIL FLOW)
# ------------------------------------------------------

def main():
    message = "Secure Email Communication"

    print("\n--- ORIGINAL MESSAGE ---")
    print(message)

    # RSA
    rsa_public, rsa_private = rsa_key_generation()
    encrypted_message = rsa_encrypt(message, rsa_public)
    decrypted_message = rsa_decrypt(encrypted_message, rsa_private)

    # ElGamal
```

```
    elgamal_public, elgamal_private = elgamal_key_generation()
    signature = elgamal_sign(message, elgamal_private, elgamal_public)
    is_valid = elgamal_verify(message, signature, elgamal_public)

    # OUTPUT
    print("\n--- RSA KEYS ---")
    print("Public Key (n, e):", rsa_public)
    print("Private Key (n, d):", rsa_private)

    print("\n--- ENCRYPTION ---")
    print("Encrypted Message:", encrypted_message)

    print("\n--- DECRYPTION ---")
    print("Decrypted Message:", decrypted_message)

    print("\n--- ELGAMAL SIGNATURE ---")
    print("Signature (r, s):", signature)

    print("\n--- VERIFICATION ---")
    print("Signature Valid:", is_valid)

# Run program
main()
```

OUTPUT:

```
--- ORIGINAL MESSAGE ---
Secure Email Communication

--- RSA KEYS ---
Public Key (n, e): (3233, 17)
Private Key (n, d): (3233, 2753)

--- ENCRYPTION ---
Encrypted Message: [2680, 1313, 281, 2160, 2412, 1313, 1992, 28, 2271, 1632, 3179, 745, 1992, 641, 2185, 2271, 2271, 2160, 2235, 3179, 281, 1632, 884, 3179, 2185, 2235]

--- DECRYPTION ---
Decrypted Message: Secure Email Communication

--- ELGAMAL SIGNATURE ---
Signature (r, s): (101, 257)

--- VERIFICATION ---
Signature Valid: True

Process finished with exit code 0
```

Question # 02

Project Used

Topic: Feistel Cipher
Course Project: Information Security Lab

---

1. Justification of Security Method

The **Feistel Cipher** is suitable for this project because:

- It allows **same structure for encryption & decryption**
- Decryption is done simply by **reversing the round keys**
- Uses **XOR**, which is reversible and efficient
- Forms the basis of strong real-world ciphers like **DES**

Compared to simple substitution or transposition ciphers, Feistel offers:

- Better diffusion
- Multiple rounds for increased security
- Practical implementation for academic projects

---

2. One Possible Vulnerability

Weak Round Function (F-Function)

If the round function uses:

- Simple XOR
- No substitution boxes (S-boxes)
- Few rounds

Then the cipher becomes vulnerable to:

- **Known-plaintext attacks**
- **Differential crypt-analysis**

An attacker could analyze patterns between plain-text and cipher-text to recover keys.

---

3. One Realistic Security Improvement

Suggested Improvement: Stronger Round Function

How it works:

- Introduce **non-linear operations** (S-boxes)
- Increase number of rounds
- Use a stronger key scheduling algorithm

This improves:

- Confusion
- Resistance to crypt-analysis
- Overall encryption strength
